

What Is JUnit 5?

- **JUnit 5** is a testing framework for writing **unit tests** in **Java**
- It provides **annotations**, **assertions**, and **tools** for structured **test development**
- Designed to **run** on the **JVM** and **integrate with** major **IDEs** and **build tools**
- Supports **writing** clean, maintainable, and readable **test code**
- Fully **replaces older JUnit versions** while **remaining backward compatible** through its architecture

 **by Rodrigo Martins Pagliares**



JUnit Platform and Jupiter API

JUnit Platform is the foundation for launching and running test frameworks on the JVM

- Supported by IntelliJ, Eclipse, NetBeans, VSCode, Maven, Gradle, and Ant
- Acts as the engine responsible for discovering and executing test classes

JUnit Jupiter API introduces a new programming and extension model

- Provides the annotations (`@Test`, `@BeforeEach`, etc.) and assertions we use
- Allows developers to create custom extensions when needed



Together, these components enable writing and running **Java unit tests**



JUnit Vintage Engine

Backward Compatibility

JUnit Vintage is the test engine for **executing older JUnit 3 and JUnit 4 tests**

Legacy Support

Ensures **backward compatibility** so legacy projects can run alongside new Jupiter tests

Gradual Migration

Developers can maintain mixed codebases during gradual framework migration

Vintage + Jupiter + Platform form the complete **JUnit 5 architecture**

Conclusion

- JUnit 5 is built from three components: Platform, Jupiter, and Vintage
- Jupiter provides modern testing capabilities, while Vintage preserves compatibility
- The platform ties everything together and integrates with major development tools
- This architecture makes JUnit 5 flexible, extensible, and future-proof





Open Floor Discussion

Have you worked with older JUnit versions before JUnit 5?

Do you plan to migrate legacy JUnit 4 tests to Jupiter?

What benefits do you see in the modular architecture of JUnit 5?